Support Ticket Triage Agent — Problem Definition, Data Processing & Evaluation Criteria

Sep 28, 2026 · @hardik chhatwal

Problem definition

Support teams read, categorize, prioritize and route every incoming ticket by hand. That is slow and inconsistent, and it delays resolution. Many tickets are routine questions the help center already answers.

This project builds an agent that makes three bounded decisions for a single ticket:

1. **Classify** it by category and priority.
2. **Route** it to the correct destination team.
3. **Resolve or escalate:** auto-resolve it with a cited knowledge-base (KB) article and a drafted reply, or flag it for a human.

Who benefits. Support agents get tickets pre-sorted and routine ones pre-answered. Customers with routine questions get an answer faster. Team leads get consistent routing.

What success looks like. Most tickets reach the right team on the first try, and routine questions are resolved from the KB. Above all, the agent almost never tells a customer a problem is solved when it isn't. That last point is the key safety requirement: a wrong auto-resolve costs far more than an unnecessary escalation.

Scope, inputs and outputs

The agent takes one ticket in and returns one decision object. Scope is deliberately narrow so every output can be checked against a label.

Input: the free text of a single support ticket (1–10,000 characters). No customer history, attachments or metadata.

Output: one `TicketDecision`:

Field	Values	Notes
`category`	billing, technical, account, feature_request, general	From the classifier

Field	Values	Notes
`priority`	low, medium, high, urgent	From the classifier
`team`	billing, technical_support, account_management, product, general_support	From the router
`action`	auto_resolve, escalate	After the safety guardrail
`kb_citation`	KB article id, or null	Required for auto_resolve
`confidence`	0.0–1.0	Model-reported match confidence
`draft_resolution`	text, or null	Only for auto_resolve, grounded in the cited article
`reasoning`	text	Short rationale for audit

Out of scope: multi-turn conversations, open-ended replies not grounded in the KB, ticket lifecycle state, and sending anything to customers.

Assumptions

- Each ticket has one primary issue. Multi-issue tickets should escalate.
- The KB is the only source of truth for auto-resolutions; the model must not answer from its own knowledge.
- A human reviews escalations, and the downstream system decides whether to send a drafted reply.

Data

Two curated, fully synthetic datasets ship in the repository: a 28-article KB corpus used for retrieval and a 76-ticket labeled set used for evaluation. Neither contains proprietary or personal data, so the project is fully reproducible.

Knowledge-base corpus (`data/kb/*.md`)

- 28 articles for a fictional SaaS product; 76–163 words each, 116 on average.
- By category: billing 9, technical 9, account 6, general 2, feature_request 2.
- Each article gives self-service steps with exact settings paths, plus explicit limits on when self-service does not apply (for example "charges older than the refund window require a billing specialist"). Those limits are what let the agent tell auto-resolvable tickets from escalations.

Labeled ticket set (`data/tickets.jsonl`)

Each line is `{id, text, category, priority, team, action}`. There are 76 tickets, 45 labeled auto_resolve and 31 escalate.

Category	Auto-resolve	Escalate	Total
billing	15	8	23
technical	13	7	20
account	9	7	16
general	5	5	10
feature_request	3	4	7
Total	**45**	**31**	**76**

Priorities: 41 low, 14 medium, 14 high, 7 urgent. Team follows category one-to-one (feature_request → product, general → general_support).

The set deliberately includes hard cases: duplicate charges, chargebacks, refunds outside the window, lost 2FA without backup codes, compromised accounts, silent outages, SSO lockouts, and multi-issue and vague tickets.

Deviation from the original spec. The spec proposed a public Kaggle or Hugging Face dataset. A curated set was used instead so every ticket is answerable, or deliberately not answerable, from this KB. The evaluator accepts any JSONL with the same fields, so a public dataset can be swapped in with `--data`.

Labeling guidelines

Each ticket's ground truth was assigned by checking it against the KB article that would answer it, using these rules.

Category, by what the ticket is fundamentally about:

- **billing:** payments, invoices, refunds, plans, subscriptions, seats, tax, discounts.
- **technical:** bugs, crashes, errors, API, webhooks, sync, import and export.
- **account:** sign-in, passwords, 2FA, email changes, SSO, account security and deletion.
- **feature_request:** asking for functionality that does not exist, or how to request it.
- **general:** support logistics, compliance, greetings and unclear asks.

Priority, by impact on the customer:

Priority	Rule	Example
urgent	Outage, data loss, security incident, or business-critical work blocked	"Every API request returns 500"
high	Core functionality broken for the user, failed payment, cannot sign in	"My card was declined this month"
medium	Degraded experience with a workaround	"Edits on my phone don't sync to my laptop"
low	Questions, cosmetic issues, feature requests	"What are your support hours?"

Team follows category: billing → billing, technical → technical_support, account → account_management, feature_request → product, general → general_support.

Action

- **auto_resolve** only when one KB article directly and completely answers the ticket.
- **escalate** when no article matches or the match is partial. Also escalate when the article itself says a specialist is needed (disputes, chargebacks, recovery without backup codes, persistent bugs, incidents), and for multi-issue, vague or emotionally escalated tickets, and for requests for features that don't exist.

Data processing pipeline

KB articles become 35 searchable chunks through five steps. Steps 1–4 run once, via `support-agent ingest`; step 5 runs on every KB search the agent makes.

1. **Parse.** Each `.md` file becomes `{article_id, title, category, body}`. The id is the filename stem (`reset_password`), the title is the `#` heading, and the category comes from the `Category:` line.
2. **Chunk.** The body is split on blank lines, and paragraphs are packed into chunks of at most 700 characters. A paragraph longer than 700 characters is hard-split with a 100-character overlap. Most articles fit in one or two chunks, so steps and their caveats stay together.
3. **Embed.** Each chunk is embedded locally with all-MiniLM-L6-v2 (384 dimensions, ONNX, Chroma's default). No API key or network call is needed after the first model download.
4. **Index.** Chunks are stored in a persistent Chroma collection using cosine distance, with ids `<article_id>::<chunk_index>` and metadata `{article_id, title, category, chunk_index}`. Re-running ingest rebuilds the collection from scratch.

5. **Retrieve.** `search_kb(query)` embeds the router's query and returns the top 4 chunks. Each hit's similarity is `1 − cosine distance`, clamped to [0, 1] and returned with the article id, title and text.

Ticket preprocessing is minimal: text is trimmed, empty tickets are rejected, and the length is capped at 10,000 characters. No other cleaning is done, so the agent sees what the customer wrote.

Model outputs are validated against the Pydantic schema. The classification and routing outputs are merged into one `TicketDecision`, and the safety guardrail runs before merging.

Evaluation criteria

Six quantitative metrics are measured against the labeled set; five have pass targets from the spec. The false auto-resolve rate is the gating safety metric.

Metric	What it measures	Target
Routing accuracy	Share of tickets sent to the correct team	⩾ 85%
Category macro-F1	Classification quality, weighting each category equally	⩾ 0.80
Priority macro-F1	Priority quality, weighting each level equally	⩾ 0.80
Escalation recall	Share of tickets that need a human that were escalated	⩾ 0.85
Escalation precision	Share of escalations that were truly needed	Reported, no target
False auto-resolve rate	Share of escalation-worthy tickets wrongly auto-resolved	**< 5%**

Definitions. N is the number of tickets. Escalation is the positive class: TP are escalate-labeled tickets the agent escalated, and FN are escalate-labeled tickets it auto-resolved.

$$\text{Routing accuracy} = \frac{\#\{\text{predicted team} = \text{labeled team}\}}{N}$$

$$\text{Macro-F1} = \frac{1}{|C|} \sum_{c \in C} \frac{2 P_c R_c}{P_c + R_c}$$

The macro-F1 average runs over every class c that appears in the labels or the predictions, with P_c and R_c the per-class precision and recall.

$$\text{Escalation recall} = \frac{TP}{TP + FN} \qquad \text{Escalation precision} = \frac{TP}{TP + FP}$$

$$\text{False auto-resolve rate} = \frac{\#\{\text{labeled escalate, predicted auto_resolve}\}}{\#\{\text{labeled escalate}\}}$$

The false auto-resolve rate equals 1 − escalation recall, so the two targets pull in the same direction. The 5% bar is the stricter one.

Qualitative criteria

- Human review of a sample of auto-resolution drafts: is the cited article the right one, and is the draft correct and grounded in it?
- Failure-mode analysis: sort mismatches into error types (ambiguous, multi-issue, priority judgement, retrieval miss) and find the most frequent.

Evaluation procedure

One command runs the real agent on every labeled ticket and scores it:

```
support-agent ingest                                    # once, builds the KB index
python eval/evaluate.py                                 # uses data/tickets.jsonl
python eval/evaluate.py --data other.jsonl --out eval/results/run.json
```

1. Load the labeled JSONL and the Chroma index. The index is built automatically if it is empty.
2. Triage each ticket with the configured model (`gpt-5.4-mini`), exactly as in production, guardrail included.
3. Compute the metrics above and print PASS or FAIL for each target.
4. Print every mismatched ticket with its expected and predicted values, for failure analysis.
5. Write the summary, the failures and every per-ticket record (with citation and confidence) to `eval/results/latest.json` .

The web UI's **Test set** tab runs the same tickets through the API and shows mismatches inline, which makes manual review of drafts and failures faster.

Reproducibility. Retrieval is deterministic. LLM outputs can vary between runs, so small metric changes (±1–2 tickets) are expected; report the run date and model with any result.

Results

All five targets pass on the 76-ticket set, and the agent made no false auto-resolves. Run of 28 September 2026, model `openai:gpt-5.4-mini`, auto-resolve threshold 0.75.

Metric	Result	Target	Status
Routing accuracy	90.8% (69 / 76)	⩾ 85%	Pass
Category macro-F1	0.851	⩾ 0.80	Pass
Priority macro-F1	0.856	⩾ 0.80	Pass
Escalation recall	1.000 (31 / 31)	⩾ 0.85	Pass
Escalation precision	0.939 (31 / 33)	none	Reported
False auto-resolve rate	**0.0% (0 / 31)**	**< 5%**	**Pass**

- All 43 of the agent's auto-resolves were on tickets labeled auto_resolve, and each cited a KB article.
- The only action errors were 2 unnecessary escalations, the safe direction: the agent escalated tickets that could have been auto-resolved.
- 60 of 76 tickets (79%) matched on all four labels.

Caveats. 76 tickets is a small sample: one more wrong routing moves accuracy by 1.3 points, and a single false auto-resolve would be 3.2%. The set was written alongside the KB, so these numbers are likely optimistic compared with real traffic. The qualitative human review of drafts has not been done yet: the metrics check that the action is right, not that the draft text is correct.

Failure-mode analysis

The 16 mismatched tickets fall into four error types. The largest is category boundaries between technical, general and feature_request; none of the errors was unsafe.

Error type	Tickets	Count	Example	Likely cause	Fix
Category boundary	T016, T034, T040, T046, T048, T049, T053, T055, T056	9	"Do you support Internet Explorer 11?" labeled technical, predicted general	Capability and how-to questions fit several category definitions; some labels are arguable (a custom rate limit read as a feature request)	Add boundary examples to the classifier prompt; review ambiguous labels
Priority judgement	T012, T018, T061, T075	4	Lost 2FA with no backup codes: labeled urgent, predicted high	Adjacent levels depend on business impact the ticket rarely states	Few-shot priority examples; score adjacent-level misses as partial credit
Over-cautious escalation	T054, T063	2	"When will the Jira integration be released?" escalated although the integrations article answers it	"When in doubt, escalate" applied to a clear match	Acceptable by design; tune only if the escalation load matters
Vague, emotional ticket	T024	1	"Nothing works… Fix it now" routed to technical_support, urgent	No concrete issue to classify; the agent guessed technical	Correctly escalated; route vague tickets to general_support in the prompt

Impact. Seven tickets reached the wrong team. Four of those (T040, T053, T055, T056) were auto-resolved with the correct article, so the wrong team label had no practical effect. Three escalations (T016, T024, T046) would need manual re-routing. The multi-issue ticket (T018) was escalated to the right team; only its priority was off.

Next steps

- [ ] Human review of a sample of the 43 auto-resolution drafts
- [ ] Tighten the category definitions with boundary examples, then re-run
- [ ] Evaluate on a larger public dataset via `--data`